CSE 405: Computer Graphics & Image Processing

Complete Questions & Answers

BSc (Engineering) in CSE, Level 4, Semester I

Examination 2024 (Jan–Jun)

Contents

Chapter 1: Introduction to Digital Image Processing

1.1 What is Digital Image Processing (DIP)?

Digital Image Processing (DIP) refers to the processing of digital images by means of a digital computer. More formally, it is the use of computer algorithms and mathematical models to process, analyze, enhance, and extract useful information from digital images.

A **digital image** is a two-dimensional function $f(x, y)$, where x and y are spatial (plane) coordinates, and the amplitude of f at any pair of coordinates (x, y) is called the **intensity** (or gray level) of the image at that point. When x, y, and the amplitude values of f are all finite, discrete quantities, the image is called a digital image.

DIP focuses on two major tasks:

1. **Improvement of pictorial information for human interpretation** — enhancing visual quality.
2. **Processing of image data for autonomous machine perception** — storage, transmission, and representation for computer analysis.

1.3 Applications of Digital Image Processing

Digital Image Processing is applied across numerous fields:

1. **Medical Imaging** — X-rays, MRI, CT scans, ultrasound image enhancement; tumor detection; surgical planning.
2. **Remote Sensing** — Satellite imagery analysis; weather forecasting; land-use classification; environmental monitoring.
3. **Industrial Automation & Robotics** — Quality inspection on assembly lines; defect detection; automated sorting and packaging.
4. **Security & Surveillance** — Face recognition; fingerprint analysis; license plate detection; CCTV footage enhancement.
5. **Agriculture** — Crop health monitoring; disease detection in plants; precision farming using drone imagery.
6. **Military & Defense** — Target detection; night vision; missile guidance; drone surveillance.
7. **Astronomy** — Enhancement of telescope images; planet/star analysis; deep-space image processing.
8. **Entertainment & Multimedia** — Photo/video editing; CGI in movies; Instagram/Snapchat filters; animation.
9. **Transportation** — Autonomous driving; traffic monitoring; number plate recognition.
10. **Forensic Science** — Crime scene image analysis; fingerprint matching; restoration of damaged photographs.

1.4 Fundamental Steps in DIP (Pipeline) — Examples

The general DIP pipeline consists of: Image Acquisition → Preprocessing → Enhancement → Restoration → Segmentation → Feature Extraction → Recognition/Classification. (See Section 5.2 and 6.5 for the full pipeline.)

Example 1: Edge Detection Pipeline

1. **Image Acquisition** — Capture the scene using a camera.
2. **Preprocessing** — Convert to grayscale; apply Gaussian smoothing to reduce noise.
3. **Enhancement** — Adjust contrast if needed (e.g., histogram equalization).
4. **Segmentation / Edge Detection** — Apply an edge detection operator (Sobel, Canny) to detect intensity discontinuities.
5. **Post-processing** — Threshold the gradient magnitude map; apply non-maximum suppression and hysteresis thresholding (Canny).
6. **Output** — Binary edge map showing object boundaries.

Example 2: Number Plate Detection Pipeline

1. **Image Acquisition** — Capture vehicle image using a road-side camera.
2. **Preprocessing** — Convert to grayscale; resize; apply noise filtering (median/Gaussian filter).
3. **Enhancement** — Histogram equalization to improve contrast of the plate region.
4. **Segmentation** — Use edge detection (Sobel/Canny) followed by morphological operations (dilation, closing) to locate the rectangular plate region. Alternatively, use thresholding.
5. **Feature Extraction** — Extract the plate region using contour detection; isolate individual characters using connected component analysis.
6. **Recognition/Classification** — Apply OCR (Optical Character Recognition) or a trained CNN to recognize each character on the plate.
7. **Output** — The detected license plate number as text.

1.5 Components of a Digital Image Processing System

A complete DIP system consists of the following components:

1. **Image Sensors** — Physical devices sensitive to the energy radiated by the object to be imaged (e.g., CCD arrays, CMOS sensors). Two elements are needed: (a) a sensing material responsive to the energy, and (b) a digitizer to convert the sensed data into digital form.

2. **Specialized Image Processing Hardware** — Dedicated hardware (e.g., digitizer boards, ALU units) that performs primitive operations such as arithmetic and logical operations on images in parallel for speed.

3. **General-Purpose Computer** — Ranges from a PC to a supercomputer depending on the application. Executes the image processing algorithms.

4. **Image Processing Software** — Specialized software modules consisting of algorithms for image acquisition, enhancement, restoration, segmentation, feature extraction, and recognition.

5. **Mass Storage** — Large storage devices (hard disks, SSDs, cloud storage) to store the large volumes of pixel data associated with images.

6. **Image Display (Monitors)** — High-resolution color monitors (LCD, LED) used to display processed images.

7. **Hardcopy Devices** — Printers, film recorders, and other devices to produce permanent copies of processed images.

8. **Networking** — Communication links (LAN, Internet) for transmitting image data between systems, especially important in remote sensing, telemedicine, and distributed processing.

Chapter 2: Digital Image Fundamentals

2.3 Image Sensing and Acquisition

How Does Image Acquisition Happen?

Image acquisition is the process of capturing a scene and converting it into a digital image. The process involves three main steps:

1. **Energy source (Illumination)** — An energy source illuminates the scene (e.g., visible light, X-rays, infrared, ultrasound). The energy is either reflected by or transmitted through the objects in the scene.
2. **Sensing element** — A sensor (e.g., CCD/CMOS array) detects the incoming energy and produces a continuous electrical signal proportional to the energy intensity at each point.
3. **Digitization** — The continuous signal is digitized by:
    - **Sampling** — Discretizing the spatial coordinates (x, y) into a finite grid of $M \times N$ pixels.
    - **Quantization** — Discretizing the amplitude (intensity) into a finite number of gray levels (e.g., 2^k levels, typically $k = 8$ giving 256 levels).

The result is a digital image represented as an $M \times N$ matrix of integer values.
Types of Image Sensors:

- **Single sensor** — One photodiode; image formed by mechanical scanning (e.g., flatbed scanner).
- **Line sensor (1-D array)** — A row of sensors; image formed by relative motion (e.g., satellite push-broom scanner).
- **Array sensor (2-D array)** — An $M \times N$ grid of sensors captures the entire image at once (e.g., digital camera CCD/CMOS chip).

Simple Image Formation Model

An image can be modeled as a two-dimensional function:

$$f(x, y) = i(x, y) \cdot r(x, y)$$

where:

- $f(x, y)$ = intensity of the image at point (x, y)
- $i(x, y)$ = **illumination component** — the amount of source illumination incident on the scene at (x, y). It is determined by the light source. Range: $0 < i(x, y) < \infty$.
- $r(x, y)$ = **reflectance component** — the fraction of illumination reflected by the object at (x, y). It is determined by the characteristics of the object. Range: $0 < r(x, y) < 1$ (where 0 = total absorption and 1 = total reflectance).

Therefore, $f(x, y)$ is bounded:

$$0 < f(x, y) < \infty$$

In practice, $f(x, y)$ is represented as a discrete function:

$$f(x, y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0, N-1) \\ f(1,0) & f(1,1) & \cdots & f(1, N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1, N-1) \end{bmatrix}$$

Each element of this matrix is a pixel with an integer gray-level value in $[0, L-1]$ where $L = 2^k$.

2.5 Basic Relationships Between Pixels

Neighbors of a Pixel

A pixel p at coordinates (x, y) has the following neighbors:

4-Neighbors $N_4(p)$: The four horizontal and vertical neighbors:

$$N_4(p) = \{(x-1, y),\ (x+1, y),\ (x, y-1),\ (x, y+1)\}$$

Diagonal Neighbors $N_D(p)$: The four diagonal neighbors:

$$N_D(p) = \{(x-1, y-1),\ (x-1, y+1),\ (x+1, y-1),\ (x+1, y+1)\}$$

8-Neighbors $N_8(p)$: All 8 surrounding neighbors:

$$N_8(p) = N_4(p) \cup N_D(p)$$

Adjacency

Two pixels p and q are adjacent if they are neighbors and their intensity values satisfy some specified criterion of similarity (e.g., both belong to the same set of intensity values V).

Three types of adjacency:

- **4-adjacency**: $q \in N_4(p)$ and q has a value from V.
- **8-adjacency**: $q \in N_8(p)$ and q has a value from V.
- **m-adjacency (mixed adjacency)**: q is m-adjacent to p if:
  (a) $q \in N_4(p)$ and q has a value from V, **OR**
  (b) $q \in N_D(p)$, q has a value from V, and $N_4(p) \cap N_4(q)$ has no pixels with values from V.

  Mixed adjacency eliminates the ambiguity of 8-adjacency (prevents multiple paths).

Connectivity

Two pixels p and q are said to be **connected** if there exists a **path** between them — a sequence of adjacent pixels from p to q. The type of connectivity (4-connectivity, 8-connectivity, or m-connectivity) depends on the adjacency definition used.

For a pixel p in a set S, the set of all pixels in S that are connected to p is called a **connected component** of S. If S has only one connected component, it is called a **connected set**.

Regions and Boundaries

- **Region**: A subset R of an image is called a region if it is a connected set. Two regions R_i and R_j are said to be adjacent if their union forms a connected set.
- **Boundary (Border)**: The boundary of a region R is the set of pixels in R that have at least one neighbor *not* in R:

$$\text{Boundary}(R) = \{p \in R \mid N_4(p) \not\subseteq R\}$$

- **Edge vs. Boundary**: An edge is defined by a local intensity discontinuity (gradient), while a boundary is a global concept (the contour of a region). Ideally, edges and boundaries coincide.

Distance Measures

For pixels $p = (x_1, y_1)$ and $q = (x_2, y_2)$:

1. Euclidean Distance:

$$D_e(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

2. City-Block Distance (D_4):

$$D_4(p, q) = |x_1 - x_2| + |y_1 - y_2|$$

Pixels with $D_4 \leq r$ form a **diamond** shape centered at p.

3. Chessboard Distance (D_8):

$$D_8(p, q) = \max(|x_1 - x_2|,\ |y_1 - y_2|)$$

Pixels with $D_8 \leq r$ form a **square** shape centered at p.

Example: For $p = (3, 5)$ and $q = (7, 2)$:

$$D_e = \sqrt{(3-7)^2 + (5-2)^2} = \sqrt{16+9} = \sqrt{25} = \mathbf{5.0}$$
$$D_4 = |3-7| + |5-2| = 4 + 3 = \mathbf{7}$$
$$D_8 = \max(|3-7|, |5-2|) = \max(4, 3) = \mathbf{4}$$

Relationship: $D_8(p, q) \leq D_4(p, q)$ and $D_8(p, q) \leq D_e(p, q) \leq D_4(p, q)$ always hold.

Quiz #2 (Time: 30 Min, Total: 15 Marks)

Q1. Image Restoration vs. Image Enhancement Marks: 2+2

Image Restoration

Image restoration is a process that attempts to reconstruct or recover a degraded image by using *a priori* knowledge of the degradation phenomenon. It models the degradation mathematically and then applies the inverse process to recover the original image.

Differences from Image Enhancement

Aspect	Image Restoration	Image Enhancement
Approach	Objective — based on mathematical/probabilistic models of degradation	Subjective — based on human preferences for what looks "good"
Methodology	Models the degradation and applies the reverse process to reconstruct the original image	Uses heuristic procedures to manipulate the image for better visual perception
Orientation	General-purpose, model-driven	Problem-oriented, task-specific

Q2. Noise Modeling & Types of Noise Marks: 2+2

Noise modeling refers to the mathematical characterization of random variations (noise) that corrupt a digital image during acquisition or transmission. Understanding the statistical properties of noise is essential for designing effective restoration algorithms.

Two Common Types of Noise

1. **Gaussian Noise** — Statistical noise whose probability density function follows a normal distribution. It commonly arises from sensor electronics and is additive in nature.

2. **Salt-and-Pepper Noise** — An impulse noise where certain pixels are randomly set to the maximum (white/salt) or minimum (black/pepper) intensity values, often caused by faulty sensors or transmission errors.

Q3. Edge-Based Segmentation Marks: 2+1

Edge-based segmentation is a technique that partitions an image into meaningful regions by detecting edges — abrupt changes or discontinuities in intensity between adjacent regions. By linking detected edges, the image is segmented into distinct regions.

Two Edge Detection Operators

1. **Sobel operator** — Uses 3×3 convolution masks to approximate the first-order gradient in both horizontal and vertical directions.

2. **Canny edge detector** — A multi-stage algorithm that provides optimal edge detection with good localization and minimal spurious responses.

Q4. Laplacian Kernel Computation Marks: 2+2

Given:

$$\text{Image I} = \begin{bmatrix} 20 & 18 & 22 \\ 17 & \underline{19} & 21 \\ 16 & 20 & 18 \end{bmatrix}, \quad \text{Kernel} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Computing the Laplacian Response at Center Pixel (19)

$$\begin{aligned} L &= (-1)(20) + (-1)(18) + (-1)(22) + (-1)(17) + (8)(19) + (-1)(21) \\ &\quad + (-1)(16) + (-1)(20) + (-1)(18) \\ &= -20 - 18 - 22 - 17 + 152 - 21 - 16 - 20 - 18 = \mathbf{0} \end{aligned}$$

Explanation

The Laplacian response is **0** because $8 \times 19 = 152$ exactly equals the sum of all 8 neighbors. The center pixel equals the average of its neighbors, meaning this region is locally smooth with no edge.

How to Use for Sharpening

The sharpened image is obtained by adding the Laplacian response to the original: $g(x, y) = f(x, y) + L(x, y) = 19 + 0 = 19$. In general: $L > 0$: pixel brighter; $L < 0$: pixel darker; $L = 0$: unchanged.

Mid-Term Examination (Time: 50 Min, Total: 30 Marks)

Q1(a). Scan Conversion Marks: 2

Scan Conversion is the process of converting continuous geometric primitives (lines, circles, polygons) described mathematically into a discrete set of pixels on a raster display (*rasterization*).

Key Considerations

- **Accuracy** — Generated pixels should closely approximate the true geometric shape.
- **Efficiency** — Use only integer arithmetic for faster computation.
- **Uniform brightness** — Lines of equal length should appear equally bright regardless of slope.
- **No gaps** — The pixel sequence must be continuous.

Q1(b). Transformation & Rotation Transformation Marks: 3

Transformation in computer graphics refers to altering the position, size, or orientation of an object by applying mathematical operations to its coordinate points.

Rotation Transformation rotates an object by angle θ about the origin:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

A positive θ rotates counterclockwise; negative θ rotates clockwise.

Q1(c). Bresenham's Line Algorithm: (3,2) to (9,6) Marks: 5

Given: Start $(3, 2)$, End $(9, 6)$. $dx = 6$, $dy = 4$, $m = 0.667 < 1$. $p_0 = 2dy - dx = 2$.

Step	p_k	Pixel	Decision
Start	—	$(3, 2)$	—
1	2	$(4, 3)$	$p \geq 0$: $x{+}1, y{+}1$, $p = -2$
2	-2	$(5, 3)$	$p < 0$: $x{+}1$, $p = 6$
3	6	$(6, 4)$	$p \geq 0$: $x{+}1, y{+}1$, $p = 2$
4	2	$(7, 5)$	$p \geq 0$: $x{+}1, y{+}1$, $p = -2$
5	-2	$(8, 5)$	$p < 0$: $x{+}1$, $p = 6$
6	6	$(9, 6)$	$p \geq 0$: $x{+}1, y{+}1$, $p = 2$

Pixels: $(3, 2) \to (4, 3) \to (5, 3) \to (6, 4) \to (7, 5) \to (8, 5) \to (9, 6)$

Q2(a). Limitations of DDA Algorithm Marks: 2

- **Floating-point arithmetic** — Computationally expensive and slower than integer-only methods.
- **Round-off error accumulation** — Errors accumulate over long lines, causing pixel drift.
- **Not suitable for complex curves** — Primarily designed for straight lines.

Q2(b). Homogeneous Coordinates Marks: 3

The homogeneous coordinate representation of a 2-D point (x, y) is: $(x, y) \to (x_h, y_h, w) = (x, y, 1)$ where w is the homogeneous parameter. This allows translation, rotation, and scaling to all be expressed as 3×3 matrix multiplications, enabling composite transformations via a single matrix: $M = M_n \times \cdots \times M_1$.

Q2(c). Cohen–Sutherland Line Clipping Marks: 5

Given: $P_1(10, 30)$ to $P_2(80, 90)$; Window: $x_{\min} = 20$, $y_{\min} = 20$, $x_{\max} = 90$, $y_{\max} = 70$.

P_1: code `0001` (LEFT); P_2: code `1000` (TOP). OR $\neq 0$, AND $= 0 \Rightarrow$ clip.

Clip P_1 vs LEFT ($x = 20$): $m = 6/7$; $y' = 30 + (6/7)(10) = 38.57$. New $P_1' = (20, 38.57)$, code `0000`.

Clip P_2 vs TOP ($y = 70$): $x' = 20 + (7/6)(70 - 38.57) = 56.67$. New $P_2' = (56.67, 70)$, code `0000`.

Both `0000` $\Rightarrow$ Accept. $\boxed{(20, 38.57) \text{ to } (56.67, 70)}$

Exam Paper – Questions 3 & 4 (First Set)

Q3(a). Bits for 8-bit 512×512 Image Marks: 2

Total bits $= 512 \times 512 \times 8 =$ **2,097,152** bits $= 256$ KB

Q3(b). General Pipeline of Image Processing Marks: 3

1. **Image Acquisition** — Capturing the raw image using cameras/sensors.
2. **Image Preprocessing** — Noise removal, contrast adjustment.
3. **Image Enhancement** — Subjective manipulation for visual quality.
4. **Image Restoration** — Objective recovery using degradation models.
5. **Image Segmentation** — Partitioning into meaningful regions.
6. **Feature Extraction** — Identifying edges, textures, shapes.
7. **Recognition/Classification** — Interpreting and classifying objects.

Q3(b cont.). Histogram Equalization Marks: 5

Definition: Histogram equalization adjusts contrast by redistributing intensity values so that the output histogram is approximately uniform, using the Cumulative Distribution Function (CDF).

Given: $\{52, 55, 52, 60, 70, 70, 80, 80, 90, 55, 50, 70, 70, 70, 80, 80\}$, $N = 16$, $L = 7$.
Formula: $s_k = \text{round}((L-1) \times \text{CDF}(r_k))$

r_k	n_k	PDF	CDF	$s_k = \text{round}(6 \times \text{CDF})$	New
50	1	0.0625	0.0625	round(0.375)	**0**
52	2	0.1250	0.1875	round(1.125)	**1**
55	2	0.1250	0.3125	round(1.875)	**2**
60	1	0.0625	0.3750	round(2.250)	**2**
70	5	0.3125	0.6875	round(4.125)	**4**
80	4	0.2500	0.9375	round(5.625)	**6**
90	1	0.0625	1.0000	round(6.000)	**6**

Mapping: $50 \to 0$, $52 \to 1$, $55 \to 2$, $60 \to 2$, $70 \to 4$, $80 \to 6$, $90 \to 6$.
Note:

- **PDF** (Probability Density Function) $= n_k/N$: fraction of total pixels at each gray level.
- **CDF** (Cumulative Distribution Function): running sum of PDFs. Always increases; last value must be 1.00.

Q4(a). Common Preprocessing Techniques Marks: 2

- Noise reduction/filtering (Gaussian blur, median filter)
- Contrast enhancement (histogram equalization, contrast stretching)
- Image resizing/scaling and normalization
- Grayscale conversion
- Edge detection and sharpening
- Geometric transformations (rotation, cropping)

Q4(b). Smoothing Filters Marks: 3

A **smoothing filter** reduces noise and fine details by averaging pixel values in a neighborhood, which **blurs** the image.

1. **Mean (Box) Filter** — Average of neighborhood (3×3 kernel with all weights $= 1/9$).
2. **Gaussian Filter** — Gaussian-weighted kernel; more weight to center.
3. **Median Filter** — Median of neighborhood; effective against salt-and-pepper noise.

Q4(c). Sharpening Filter on 5×5 Image Marks: 5

Kernel (sum = 1): $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$. Zero-padding for boundary pixels.

- **10 at (1,0)** [left boundary]: $= 0 - 10 + 0 - 0 + 50 - 20 + 0 - 11 + 0 = \mathbf{9}$
- **15 at (2,2)** [interior]: $= 0 - 22 + 0 - 20 + 75 - 10 + 0 - 10 + 0 = \mathbf{13}$
- **20 at (4,3)** [bottom boundary]: $= 0 - 10 + 0 - 20 + 100 - 25 + 0 - 0 + 0 = \mathbf{45}$

Effect: Interior (15→13): dark spot made darker, increasing contrast. Boundary (20→45): dramatic increase due to artificial edge from zero-padding.

Exam Paper – Questions 1 & 2 (Second Set)

Q1(a). Spatial Filtering vs. Frequency Domain Filtering Marks: 2+3

Feature	Spatial Filtering	Frequency Domain
Domain	Directly on pixel values	Frequency spectrum (FFT)
Operation	Convolution with kernel	Multiplication in freq. domain
Speed	Faster for small kernels	Slower (FFT/IFFT overhead)
Best for	Edge detection, local smoothing	Periodic noise removal, compression

Q1(b). Bits for 256×256 Image with 256 Gray Levels Marks: 2

256 gray levels = 8 bits/pixel. Total = $256 \times 256 \times 8 = \mathbf{524{,}288}$ bits = 64 KB

Q1(c). Image Restoration & Degradation Model Marks: 5

Image restoration recovers an original image from a degraded version using mathematical models.

Degradation Model:

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$$

where f = original, h = degradation function, η = additive noise, g = degraded image. In frequency domain: $G(u, v) = H(u, v) \cdot F(u, v) + N(u, v)$.

Q2(a). Gaussian Noise Marks: 2

Gaussian noise is statistical noise whose PDF follows the normal distribution:

$$p(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

characterized by mean μ and standard deviation σ. Arises from sensor electronics, poor lighting, and high temperatures.

Q2(b). Fundamental Steps in Digital Image Processing Marks: 4

1. **Image Acquisition** — Capturing and digitizing.
2. **Image Enhancement** — Bringing out obscured details.
3. **Image Restoration** — Recovering using degradation models.
4. **Color Image Processing** — Processing color information.
5. **Wavelets & Multiresolution** — Multi-scale representation.
6. **Compression** — Reducing storage/bandwidth.

7. **Morphological Processing** — Extracting shape components.
8. **Segmentation** — Partitioning into regions/objects.
9. **Representation & Description** — Converting to features.
10. **Recognition/Classification** — Assigning labels.

Q2(c). Image Interpolation Marks: 2+2

Image interpolation estimates unknown pixel values at non-integer locations. Essential for resizing, rotating, or geometric transformations.

1. **Nearest Neighbor** — Assigns value of closest pixel. Fast but blocky.
2. **Bilinear** — Weighted average of 2×2 neighborhood. Smoother but slightly blurred.

Q2(d). Histogram Equalization on 5×5 Image Marks: 5

Given: $\begin{bmatrix} 1 & 1 & 5 & 6 & 6 \\ 3 & 1 & 1 & 5 & 6 \\ 3 & 5 & 5 & 3 & 3 \\ 3 & 1 & 5 & 1 & 3 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$, $N = 25$, $L - 1 = 6$.

r_k	n_k	PDF	CDF	s_k	New
1	11	0.44	0.44	round(2.64)	**3**
3	6	0.24	0.68	round(4.08)	**4**
5	5	0.20	0.88	round(5.28)	**5**
6	3	0.12	1.00	round(6.00)	**6**

Equalized: $\begin{bmatrix} 3 & 3 & 5 & 6 & 6 \\ 4 & 3 & 3 & 5 & 6 \\ 4 & 5 & 5 & 4 & 4 \\ 4 & 3 & 5 & 3 & 4 \\ 3 & 3 & 3 & 3 & 3 \end{bmatrix}$

Exam Paper – Questions 3 & 4 (Second Set)

Q3(a). 4-Adjacency and 8-Adjacency Marks: 3

4-Adjacency: q is a direct neighbor (top, bottom, left, right): $N_4(p) = \{(x\pm1, y),\ (x, y\pm1)\}$

8-Adjacency: All 8 surrounding neighbors including diagonals: $N_8(p) = N_4(p) \cup N_D(p)$

Q3(b). City-Block and Euclidean Distance Marks: 3

$P(15, 20)$, $Q(25, 30)$:

$$D_4 = |15 - 25| + |20 - 30| = \mathbf{20}$$
$$D_e = \sqrt{(-10)^2 + (-10)^2} = 10\sqrt{2} \approx \mathbf{14.14}$$

Q3(c). Spatial Correlation vs. Convolution Marks: 2+2

	Correlation	**Convolution**
Kernel flip	No flip	Flipped 180°
Main use	Template matching	Filtering

Q3(d). 2D Spatial Convolution Marks: 5

Given: $f = 5 \times 5$ impulse at center; $w = \begin{bmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{bmatrix}$

Step 1: Flip kernel 180°: $w' = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

Step 2: Zero-pad image to 7×7.

Step 3: Slide flipped kernel. Since f is a unit impulse, convolution produces the *original* kernel centered at the impulse:

$$f * w = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 9 & 8 & 7 & 0 \\ 0 & 6 & 5 & 4 & 0 \\ 0 & 3 & 2 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Q4(a). Image Segmentation & Importance Marks: 3

Image segmentation partitions an image into meaningful segments where pixels share similar characteristics (intensity, color, texture).

Importance:

- **Object recognition** — Isolates objects from background.
- **Boundary detection** — Critical in medical imaging, autonomous driving.
- **Simplified analysis** — Reduces raw pixels to labeled regions.

Q4(b). Laplacian vs. Sobel Operators Marks: 3

Feature	Laplacian	Sobel
Derivative	Second-order ($\nabla^2 f$)	First-order (∇f)
Kernels	1 (omnidirectional)	2 (G_x, G_y)
Noise	Highly sensitive	Moderately sensitive
Edge type	Zero-crossings	Gradient magnitude peaks

Q4(c). Laplacian Filter for Edge Enhancement Marks: 4

Masks: 4-connected: $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ 8-connected: $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ (both sum $= 0$).

How it works: $\nabla^2 f =$ (center weight) $\times f(x, y) -$ sum of neighbors. Output $= 0$ in flat regions, large at edges.

Sharpening: $g(x, y) = f(x, y) + c \cdot \nabla^2 f(x, y)$. Alternatively, combine into single kernel: center $= 5$ (4-conn.) or 9 (8-conn.), sum $= 1$.

Q4(d). Thresholding and Region Growing Marks: 5

Thresholding

Simplest segmentation — separates pixels by intensity vs. threshold T:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) \geq T \\ 0 & \text{if } f(x, y) < T \end{cases}$$

Global thresholding: one T for entire image. **Otsu's method:** auto-selects optimal T.

Region Growing

Starts from seed points, iteratively adds similar connected neighbors:

1. Select seed point(s).
2. Examine 4 or 8 neighbors.
3. Add if similarity criterion is met (e.g., $|f(x, y) - \mu_R| < \Delta$).
4. Repeat until no more pixels qualify.

	Thresholding	Region Growing
Approach	Global, all pixels	Local, grows from seed
Connectivity	No guarantee	Connected regions only
Input	Threshold T	Seed + similarity criterion

Chapter 5: Image Degradation, Restoration & Noise Filters

Ch5 Q1. Model of Image Degradation and Restoration Marks: 5

(See Section 6.3 for the detailed answer.)

The degradation/restoration process is modeled as:

$$g(x, y) = h(x, y) * f(x, y) + \eta(x, y)$$

- $f(x, y)$ = original (undegraded) image
- $h(x, y)$ = degradation function (e.g., motion blur, defocus, atmospheric turbulence)
- $\eta(x, y)$ = additive noise (Gaussian, salt-and-pepper, etc.)
- $g(x, y)$ = observed degraded image
- $*$ = convolution operator

In the **frequency domain**:

$$G(u, v) = H(u, v) \cdot F(u, v) + N(u, v)$$

The **goal of restoration** is to estimate $\hat{f}(x, y)$ (or $\hat{F}(u, v)$) given knowledge (or estimates) of H and N. Common restoration filters include the Inverse filter, Wiener filter, and Constrained Least Squares filter.

Ch5 Q2. Gaussian Noise and Salt-and-Pepper Noise Marks: 3

(See Section 6.4 for the detailed answer.) and 3.2 for the core definitions.

Gaussian Noise

PDF:

$$p(z) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

- Bell-shaped curve centered at mean μ.
- Spread controlled by standard deviation σ.
- Arises from electronic circuit noise, sensor thermal noise.
- Affects *all* pixels — every pixel has a small random intensity change.

Salt-and-Pepper Noise

PDF:

$$p(z) = \begin{cases} P_a & \text{if } z = a \text{ (pepper, dark)} \\ P_b & \text{if } z = b \text{ (salt, bright)} \\ 0 & \text{otherwise} \end{cases}$$

- Random pixels are set to either minimum ($a = 0$, pepper/black) or maximum ($b = 255$, salt/white).
- Caused by faulty sensors, memory errors, or transmission bit errors.
- Affects only *some* pixels — appears as scattered black/white dots.
- Best removed by **median filter**.

Ch5 Q3. Mean Filters with Equations Marks: 3

Let S_{xy} denote the set of coordinates in an $m \times n$ rectangular window centered at (x, y).

1. Arithmetic Mean Filter

$$\hat{f}(x,y) = \frac{1}{mn} \sum_{(s,t)\in S_{xy}} g(s,t)$$

Computes a simple average. Smooths uniform noise but blurs edges.

2. Geometric Mean Filter

$$\hat{f}(x,y) = \left[\prod_{(s,t)\in S_{xy}} g(s,t) \right]^{\frac{1}{mn}}$$

Achieves smoothing comparable to the arithmetic mean but **loses less image detail**. Effective for Gaussian noise.

3. Harmonic Mean Filter

$$\hat{f}(x,y) = \frac{mn}{\sum_{(s,t)\in S_{xy}} \frac{1}{g(s,t)}}$$

Works well for **salt noise** and Gaussian noise, but **fails for pepper noise** (division by zero when pixel = 0).

4. Contra-Harmonic Mean Filter

$$\hat{f}(x,y) = \frac{\sum_{(s,t)\in S_{xy}} g(s,t)^{Q+1}}{\sum_{(s,t)\in S_{xy}} g(s,t)^{Q}}$$

where Q is the **order** of the filter.

- $Q > 0$: eliminates **pepper** noise (dark pixels).
- $Q < 0$: eliminates **salt** noise (bright pixels).
- $Q = 0$: reduces to arithmetic mean filter.
- $Q = -1$: reduces to harmonic mean filter.
- Cannot eliminate both salt *and* pepper simultaneously.

Ch5 Q4. Noise Removing Techniques in Computer Vision Marks: 5

Spatial Domain Techniques

1. **Mean (Averaging) Filter** — Replaces each pixel with the average of its neighborhood. Simple but blurs edges.
2. **Median Filter** — Replaces each pixel with the median of its neighborhood. Excellent for salt-and-pepper noise; preserves edges.
3. **Gaussian Filter** — Convolves with a Gaussian kernel. Effective for Gaussian noise; adjustable via kernel size and σ.
4. **Bilateral Filter** — Considers both spatial distance and intensity difference. Smooths noise while preserving edges.

Frequency Domain Techniques

1. **Low-Pass Filtering** — Attenuates high-frequency noise components using ideal, Butterworth, or Gaussian low-pass filters.
2. **Wiener Filter** — Optimal linear filter minimizing mean square error. Requires knowledge of noise and signal power spectra.
3. **Wavelet Denoising** — Decomposes image into multi-scale wavelets, thresholds noisy coefficients, and reconstructs. Preserves edges well.

Deep Learning-Based

1. **CNNs (e.g., DnCNN)** — Learn noise patterns from training data; often outperform traditional methods.
2. **Autoencoders** — Encoder-decoder networks trained to map noisy images to clean ones.

Chapter 9: Morphological Operations

Ch9 Q1. Morphological Operations Marks: 2

Morphological operations are image processing techniques that process images based on their shapes using a small shape called a **structuring element**. They are primarily applied to binary images and are used to extract, modify, or analyze geometric structures. The fundamental operations are **erosion** and **dilation**, from which more complex operations (opening, closing, etc.) are derived.

Ch9 Q2. Structuring Element Marks: 2

A **structuring element** (SE) is a small binary matrix (shape/probe) used to interact with the image during morphological operations. It has a defined shape (square, cross, disk, line, etc.) and a designated **origin** (center point).

The structuring element acts as a "probe" — it is placed at every pixel of the image, and the morphological operation checks how the SE fits or intersects with the foreground pixels. The choice of SE shape and size directly determines the effect of the operation.

Ch9 Q3. Erosion Marks: 5

Erosion shrinks the foreground objects in a binary image. The output pixel is set to 1 only if the structuring element *completely fits* inside the foreground at that location.

Equation:

$$A \ominus B = \{z \mid (B)_z \subseteq A\}$$

where A is the input image, B is the structuring element, and $(B)_z$ is B translated to point z.

In words: Erosion of A by B is the set of all points z such that B, translated by z, is completely contained in A.

Effects:

- Foreground objects **shrink**; boundaries retract inward.
- Small objects and thin protrusions are **removed**.
- Gaps between objects **widen**; overlapping objects become separated.

Ch9 Q4. Dilation Marks: 5

Dilation expands the foreground objects. The output pixel is set to 1 if the structuring element *overlaps at least one* foreground pixel at that location.

Equation:

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\}$$

where $\hat{B}$ is the reflection of B (rotated 180°).

In words: Dilation of A by B is the set of all points z such that the reflection of B, translated by z, has at least one element in common with A.

Effects:

- Foreground objects **grow**; boundaries expand outward.

- Small holes and gaps are **filled**.
- Nearby objects may **merge** together.

Note: Dilation and erosion are **duals**: $(A \ominus B)^c = A^c \oplus \hat{B}$

Ch9 Q6. Opening and Closing Marks: 6

Opening ($A \circ B$)

Opening is **erosion followed by dilation** with the same structuring element:

$$A \circ B = (A \ominus B) \oplus B$$

Effects:

- Smooths object contours.
- **Removes** small objects, thin bridges, and protrusions smaller than the SE.
- Does *not* significantly change the size/shape of larger objects.
- Is **anti-extensive**: $A \circ B \subseteq A$ (never adds pixels beyond the original).
- Is **idempotent**: $(A \circ B) \circ B = A \circ B$.

Closing ($A \bullet B$)

Closing is **dilation followed by erosion**:

$$A \bullet B = (A \oplus B) \ominus B$$

Effects:

- Smooths object contours.
- **Fills** small holes, narrow gaps, and thin breaks.
- Fuses nearby objects that are close together.
- Is **extensive**: $A \subseteq A \bullet B$ (never removes pixels from the original).
- Is **idempotent**: $(A \bullet B) \bullet B = A \bullet B$.

Duality

Opening and closing are duals of each other:

$$(A \bullet B)^c = A^c \circ \hat{B}$$

Importance in Image Processing

- **Noise removal:** Opening removes small bright spots (salt noise); Closing fills small dark holes (pepper noise).
- **Shape analysis:** Extract smooth object boundaries for measurement and recognition.
- **Pre/post-processing:** Often used before segmentation or feature extraction to clean up binary images.
- **Text processing:** Separate/connect characters in document image analysis.

Chapter 10: Image Segmentation

Ch10 Q1. Image Segmentation Definition and Importance Marks: 4

(See Section 7.5 for the detailed answer.) for the core definition and importance.

Additional details: Segmentation subdivides an image into its constituent regions or objects. The level of subdivision depends on the problem being solved — segmentation should stop when the objects or regions of interest have been isolated. Segmentation algorithms are generally based on two properties of intensity values:

1. **Discontinuity** — Partitioning based on abrupt changes (edges, lines, points).
2. **Similarity** — Partitioning based on regions with similar properties (thresholding, region growing, region splitting/merging).

Ch10 Q1b. Point Detection Using Laplacian Kernel

Point detection identifies isolated points in an image — single pixels that differ significantly from their surroundings. This is a special case of discontinuity-based segmentation.

Method

A point is detected using the **Laplacian kernel** (second-order derivative). The Laplacian of an image $f(x, y)$ is:

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y)$$

The standard Laplacian kernel for point detection is:

$$\text{4-connected: } \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}, \quad \text{8-connected: } \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Detection Criterion

1. Convolve the image with the Laplacian kernel to get the response $R(x, y)$.
2. A point is detected at (x, y) if:

$$|R(x, y)| \geq T$$

where T is a non-negative threshold.

Why it works: In a flat (uniform) region, the Laplacian response is zero because the center pixel equals its neighbors. At an isolated bright/dark point, the center pixel differs greatly from all neighbors, producing a large $|R|$ value. The threshold T filters out small responses from noise.

Types of Segmentation Based on Discontinuity

Type	Detects	Kernel/Method
Point Detection	Isolated pixels	Laplacian ($\nabla^2 f$); threshold $\lvert R\rvert \geq T$
Line Detection	Thin lines (horizontal, vertical, diagonal)	Four directional 3×3 kernels (sum = 0)
Edge Detection	Boundaries between regions	Sobel (1st derivative), Canny, Laplacian (2nd derivative)

Ch10 Q2. Line Detection Kernels (Four Directions) Marks: 6

Line detection uses convolution kernels tuned to respond maximally to lines at specific orientations. The four standard kernels for single-pixel-wide lines are:

Horizontal Line Detector

$$\begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

Responds maximally to **horizontal** lines (row of bright pixels between dark rows).

Vertical Line Detector

$$\begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$$

Responds maximally to **vertical** lines.

+45° Line Detector

$$\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

Responds maximally to lines at +45° (top-left to bottom-right diagonal).

−45° Line Detector

$$\begin{bmatrix} -1 & -1 & 2 \\ -1 & 2 & -1 \\ 2 & -1 & -1 \end{bmatrix}$$

Responds maximally to lines at −45° (top-right to bottom-left diagonal).

Process

1. Apply all four kernels to the image via convolution.
2. The absolute response $|R|$ at each pixel indicates the strength of a line at that orientation.
3. Compare responses: if $|R_i| > |R_j|$ for all $j \neq i$, pixel belongs to a line in direction i.
4. All four kernels have sum $= 0$, so flat regions produce zero response.
5. Thresholding the response map isolates the detected lines.

Ch10 Q3. Thresholding Marks: 1

(See Section 7.8 for the detailed answer.) for the full answer.

Summary: Thresholding segments an image by comparing each pixel to a threshold T: pixels $\geq T$ become foreground (1), pixels $< T$ become background (0). Global thresholding uses one T; adaptive thresholding computes local thresholds. Otsu's method automatically finds the optimal T.

Ch10 Q5. Importance of Noise in Image Thresholding Marks: 5

Noise significantly degrades the performance of thresholding-based segmentation:

1. **Histogram degradation** — Noise broadens and flattens the peaks in the image histogram, making it difficult to identify a clear valley (threshold point) between object and background classes.
2. **Misclassification** — Random intensity fluctuations push pixels across the threshold boundary, causing foreground pixels to be classified as background and vice versa (false positives/negatives).
3. **Fragmented regions** — In the presence of noise, a single object may appear as multiple disconnected fragments after thresholding, breaking spatial coherence.
4. **Otsu failure** — Automatic methods like Otsu's assume a bimodal histogram. Heavy noise makes the histogram unimodal, causing Otsu to select a meaningless threshold.
5. **Mitigation strategies:**
   - Apply smoothing (Gaussian, median filter) *before* thresholding to reduce noise.
   - Use adaptive/local thresholding instead of global.
   - Apply morphological opening/closing *after* thresholding to clean up noise-induced artifacts.

Ch10 Q6a. Region Growing Marks: 2

(See Section 7.8 for the detailed answer.) for the core definition.

Summary: Region growing is a pixel-based segmentation that starts from seed points and iteratively appends spatially adjacent pixels that satisfy a similarity predicate. It guarantees connected regions and is robust to noise.

Ch10 Q6b. Region Growing Algorithm Marks: 5

Given: $f(x, y)$ = input image; $S(x, y)$ = seed array (1's at seed locations, 0's elsewhere); Q = predicate at each (x, y). Arrays f and S are the same size.

Basic Region Growing Algorithm:

1. **Find all connected components** in $S(x, y)$ and label them. Erode each component to a single pixel to get individual seed points. Let $S_1, S_2, \ldots, S_n$ denote the n seed points.
2. **For each seed point** S_k **(**$k = 1, 2, \ldots, n$**):**
   a. Initialize region R_k to contain only the seed pixel S_k.
   b. Examine all 8-connected (or 4-connected) unassigned neighbor pixels of the current region R_k.
   c. For each neighbor pixel at (x, y): if predicate $Q(x, y)$ is TRUE (e.g., $|f(x, y) - \text{mean}(R_k)| < T$ for some threshold T), append (x, y) to region R_k.
   d. Repeat step (b)–(c) with the updated region R_k until no more pixels can be added.
3. **Label all remaining unassigned pixels** (those where Q was never TRUE) as background or assign to the nearest region.
4. **Stop** when every pixel has been assigned to a region or to the background.

Properties:

- $\bigcup_{k=1}^{n} R_k$ covers the entire image region of interest.
- $R_i \cap R_j = \emptyset$ for $i \neq j$ (regions are disjoint).
- $Q(R_k) =$ TRUE for all k (all pixels in a region satisfy the predicate).
- $Q(R_i \cup R_j) =$ FALSE for adjacent regions $i \neq j$ (distinct regions differ).

Chapter 11: Representation & Description

Ch11 Q1. Boundary Preprocessing Marks: 2

Boundary preprocessing refers to the set of operations applied to a raw boundary (extracted from segmentation) to prepare it for further representation and description. The raw boundary is often noisy, irregular, and excessively detailed, so preprocessing simplifies it while preserving essential shape information.

Key boundary preprocessing techniques:

- **Boundary resampling** — Reducing the number of boundary points by sampling at uniform intervals. This smooths out local noise and reduces computation.
- **Chain code smoothing** — Walking along the boundary and recording directional codes (4-directional or 8-directional). Normalization is done by selecting the smallest magnitude integer as the starting point to make the code rotation-invariant.
- **First difference of chain code** — Computing the difference between consecutive chain code elements (modulo n, where $n = 4$ or 8). This makes the representation **rotation-invariant**.
- **Boundary smoothing** — Applying a low-pass filter or polynomial fitting to the boundary coordinates to remove small fluctuations.

Ch11 Q2. Boundary Following Algorithm (Moore-Neighbor Tracing) Marks: 6

Goal: Given a binary image with a region R of 1-valued pixels, trace the complete outer boundary of R and return an ordered list of boundary pixels.

Setup and Notation

- The image contains a region R (pixels with value 1) and background (value 0).
- The **Moore neighborhood** of a pixel p consists of the 8 surrounding pixels (8-connectivity).
- b = current boundary pixel; c = the pixel from which b was entered (backtrack pixel).

Algorithm Steps

1. **Find the starting pixel (b_0):**
   - Scan the image from top-left, row by row, left to right.
   - The first 1-valued pixel encountered is b_0 (the starting boundary pixel).
   - Set c_0 = the pixel to the *left* of b_0 (this is a 0-pixel from the background, since we scanned from the left).
2. **Set current boundary pixel $b = b_0$ and backtrack pixel $c = c_0$.**

3. **Examine the Moore neighborhood of b:**
   - Start from pixel c (the backtrack pixel) and examine pixels in a **clockwise** direction around b.
   - The first 1-valued pixel found in this clockwise search becomes the **next boundary pixel**.
4. **Update:**
   - Let the found pixel be the new b.
   - Let c = the background pixel that was examined just *before* finding the new b (i.e., the last 0-pixel in the clockwise search).
   - Add the new b to the boundary list.
5. **Stopping criterion:** Repeat steps 3–4 until the algorithm returns to the starting pixel b_0 *and* enters it from the same direction c_0. That is, stop when $b = b_0$ **and** $c = c_0$.
6. **Output:** The sequence of boundary pixels $b_0, b_1, b_2, \ldots$ forms the ordered boundary of region R.

Illustration with the Given Figure (Steps A–F)

The figure shows the first few steps of the boundary following algorithm on a binary region:

Step	Description
A	The binary region R (all 1-valued pixels) is shown. Scanning from top-left, the first 1-pixel is found: this becomes b_0. c_0 is to its left.
B	At b_0: search clockwise from c_0. First 1-pixel found $\rightarrow$ new $b = b_1$. Update c to the 0-pixel just before b_1.
C	At b_1: search clockwise from new c. First 1-pixel found $\rightarrow b_2$.
D	At b_2: search clockwise from c. First 1-pixel found $\rightarrow b_3$.
E	Continuing the same process... each step finds the next boundary pixel by clockwise Moore-neighborhood search from the backtrack pixel.
F	The algorithm continues until it returns to b_0 with the same entry direction c_0. The shaded pixels show the complete traced boundary.

Key properties:

- The algorithm traces the **outer boundary** in clockwise order.
- It handles regions of any shape, including those with concavities.
- The stopping condition ($b = b_0$ **and** $c = c_0$) prevents premature termination for regions where the start pixel is revisited from a different direction.
- Time complexity: $O(n)$ where n is the number of boundary pixels.

Ch11 Q3. Minimum Perimeter Polygon (MPP) Algorithm Marks: 5

The **Minimum Perimeter Polygon (MPP)** is a polygonal approximation of a boundary that yields the polygon with the **smallest possible perimeter** that still encloses the original boundary region.

Key Concepts

- The boundary is enclosed within a **cellular complex** — a grid of square cells. The boundary passes through cells, creating an inner wall and an outer wall.
- Two types of vertices arise at cell corners:
  - **Convex vertices (white, W)** — located on the outer wall.
  - **Concave vertices (black, B)** — located on the inner wall, with corresponding **mirror vertices** diagonally opposite on the outer wall.
- The MPP vertices coincide with either convex or concave (mirrored) vertices.

MPP Algorithm

Initialization: Set $WC = BC = V_L$, where V_L is the first vertex in the vertex list.

For each subsequent vertex V_K in the list:

Compute the **sign function** (cross-product test) to determine which side of a line a point lies on:

$$\text{sgn}(P_1, P_2, P_3) = (P_2 - P_1) \times (P_3 - P_1)$$

- > 0: P_3 is to the **left** (positive side) of line P_1P_2.
- < 0: P_3 is to the **right** (negative side).
- $= 0$: P_3 is **on** the line (collinear).

Decision rules for each new vertex V_K:

1. If V_K is on the **positive side** of line (V_L, WC):
   - WC (white candidate) "crawls" forward: update $WC = V_K$.
2. If V_K is on the **negative side** of line (V_L, BC):
   - BC (black candidate) "crawls" forward: update $BC = V_K$.
3. If V_K is on the **negative side** of line (V_L, WC):
   - WC becomes a new **MPP vertex**.
   - Set $V_L = WC$, and reinitialize: $WC = BC = V_L$.
   - Continue with the next vertex after V_L.
4. If V_K is on the **positive side** of line (V_L, BC):
   - BC becomes a new **MPP vertex**.

- Set $V_L = BC$, and reinitialize: $WC = BC = V_L$.
- Continue with the next vertex after V_L.

Termination: The algorithm terminates when it reaches the first vertex again, having processed all vertices. The collected V_L vertices are the **vertices of the MPP**.

Intuition

Think of the MPP as a rubber band stretched around the boundary. The rubber band tries to snap to the tightest fit, touching only the convex/concave corners. The WC tracks the outermost (convex) candidate and BC tracks the innermost (concave) candidate. When a vertex "crosses" a candidate line, that candidate is confirmed as an MPP vertex.

Ch11 Q4. Texture Marks: 3

Texture in image processing refers to the spatial arrangement of intensity values (or colors) in an image region that provides information about the surface structure, patterns, and visual feel of objects. It quantifies properties such as smoothness, coarseness, regularity, and directionality.

Key Characteristics

- Texture describes **local spatial variation** of pixel intensities, not the overall brightness.
- A region has a **constant texture** if a set of local statistics or other local properties are constant, slowly varying, or approximately periodic.
- Examples: grass, sand, brick wall, fabric weave, tree bark.

Approaches to Texture Analysis

1. **Statistical approach** — Analyzes the statistical distribution of pixel intensities and their spatial relationships. Methods include:
   - **Gray-Level Co-occurrence Matrix (GLCM)** — Measures how often pairs of pixels with specific intensity values occur at a given spatial relationship. Features: contrast, correlation, energy, homogeneity.
   - **Histogram-based features** — Mean, variance, skewness, kurtosis, entropy of intensity distributions.
2. **Structural approach** — Describes texture as a set of primitive elements (texels) arranged according to placement rules. Works well for regular, repeating patterns (e.g., brick wall, checkerboard).
3. **Spectral approach** — Uses Fourier transform to detect periodicity and directionality. High-energy narrow peaks in the spectrum indicate dominant texture orientations.

Appendix: Kernel Identification Cheat Sheet

The Golden Rule: Sum of All Kernel Elements

Sum	Type	What to Do
$= 0$	Edge Detection	Output = edges only. Sharpen: $g = f + L$ (two steps)
$= 1$ (negatives)	Sharpening	Output = sharpened image (one step)
$= 1$ (positive)	Smoothing	Output = blurred image (one step)

Relationship

Sharpening (sum=1) = Identity (sum=1) + Laplacian (sum=0)

Center weights: $4 + 1 = 5$ (4-conn. sharpen), $8 + 1 = 9$ (8-conn. sharpen).